MoodAnswer AI

Interview Explanation and Working Guide

Node.js + Express + EJS MVC project with mood-aware AI chat, rooms, games, voice, history, MongoDB, Joi validation, and adaptive UI.

Project Type	Adaptive Mood-Aware Conversational Assistant
Backend	Node.js, Express.js, EJS, Express Router
Database	MongoDB with Mongoose models
AI Layer	Google Gemini API with OpenRouter fallback support
Interview Goal	Explain architecture, working flow, features, and decisions clearly

One-line pitch: MoodAnswer AI is an AI assistant platform that answers user questions in a tone adapted to the user's selected mood, while also supporting FunTalk, AI Rooms, FunGames, voice features, history, and adaptive themes.

1. Interview Opening Script

If an interviewer asks, 'Tell me about your project', use this simple explanation:

I built MoodAnswer AI, a full-stack Node.js and Express project where users can log in, select their current mood, ask questions, and receive AI-generated answers in a tone that matches their mood. The app uses EJS for server-side pages, MongoDB with Mongoose for users and answer history, Joi for validation, sessions for authentication, and a service layer to call Gemini/OpenRouter AI APIs. I structured it using MVC so routes, controllers, services, models, middleware, and validators are cleanly separated.

Short Hinglish version for confidence:

Mera project MoodAnswer AI hai. Isme user login karta hai, apna mood choose karta hai, question ask karta hai, aur AI us mood ke hisaab se answer ka tone change karta hai. Backend Express/EJS MVC architecture me hai, database MongoDB hai, validation Joi se hoti hai, password bcrypt se secure hota hai, aur AI response Gemini API se generate hota hai.

Main project goals:

- Give correct AI answers with mood-based tone control.
- Save user history so previous answers can be reviewed.
- Provide specialized modes: MoodAnswer, FunTalk, AI Rooms, and FunGames.
- Use a professional Express MVC structure suitable for scaling.
- Keep validation, authentication, business logic, and UI separate.

2. Problem Statement and Solution

Problem:

Normal Q&A; assistants usually answer the question but do not adapt their explanation style to how the user feels. A confused student may need a step-by-step explanation, while a tired user may need a short answer. A stressed user may need a calm and compact reply.

Solution:

MoodAnswer AI asks for the user's mood and answer preferences, sends that context to the AI service, and returns a structured response with answer, example, supportive line, optional safe humor, quick actions, and metadata for history.

Mood-aware Q&A; Answers become simple, calm, short, detailed, or supportive based on mood.	**Adaptive Rooms** Study Room, Coding Help, Chill Cafe, Gaming Lounge, Meme Zone, and more.	**History** Every successful answer is saved with question, mood, settings, metadata, and timestamp.
Voice Features Voice input and answer read-aloud support in the chat interface.	**Safe Humor** Light jokes only for safe topics; skipped for serious or sensitive questions.	**Clean Architecture** Express Router, controllers, services, Mongoose models, Joi validators, middleware.

3. Tech Stack Explanation

Technology	Where Used	Interview Explanation
Node.js	Runtime	Runs the backend JavaScript server.
Express.js	Backend framework	Handles routes, middleware, sessions, static files, APIs, and page rendering.
EJS	View engine	Renders dynamic HTML pages using res.render().
Express Router	routes/	Keeps page routes and API routes modular instead of writing everything inside app.js.
MongoDB + Mongoose	models/	Stores users and saved answers using schemas and models.
Joi	validators/	Validates login, signup, answer requests, and history query inputs.
bcrypt	utils/password.js	Hashes and verifies passwords securely.
express-session	app.js	Keeps logged-in user session on the server side.
dotenv	config/env.js	Loads API keys, database URI, port, and session secret from .env.
morgan	app.js	Logs HTTP requests during development.
method-override	app.js	Allows HTML forms to support PUT/DELETE in future features.

4. High-Level Architecture

Main request flow:

```
Browser
  |
  v
Express app.js
  |
  +-- Page Routes -> Page Controller -> res.render(EJS view)
  |
  +-- API Routes -> Joi Validation -> Auth Middleware
          |
          v
       Controller
          |
          v
       Service Layer
          |
          +-- Mongoose Models -> MongoDB
          |
          +-- AI Service -> Gemini/OpenRouter API
          |
          v
       JSON response back to frontend
```

Why this architecture is good:

- app.js remains clean and only configures the application.
- routes/ only defines endpoints and middleware chain.
- controllers/ handle request and response logic.
- services/ contain reusable business logic and external API calls.
- models/ define MongoDB structure through Mongoose schemas.
- validators/ keep Joi validation separate from controllers.
- middleware/ handles authentication, validation, and error processing.

5. Folder Structure and Responsibility

```
project-root/
  app.js                  Express configuration and server startup
  config/                 Environment and database setup
  routes/                 Express Router files
  controllers/            Request/response logic
  services/               Business logic and external API calls
  models/                 Mongoose schemas
  middleware/             Auth, validation, error middleware
  validators/             Joi schemas
  utils/                  Shared helpers
  views/                  EJS pages and partials
  public/                 Static CSS, browser JS, assets, sounds, JSON data
```

File/Folder	Purpose
app.js	Sets EJS, sessions, cookies, JSON parsing, method override, static files, routes, and errors.
routes/apiRoutes.js	Defines /api/login, /api/signup, /api/logout, /api/session, /api/answer, /api/history.
routes/pageRoutes.js	Defines pages like /login, /signup, /, /ask, /chat, /rooms, /history.
controllers/answerController.js	Handles answer generation request and history response.
services/aiService.js	Builds AI prompts, calls Gemini/OpenRouter, parses output.
models/User.js	User schema with username, email, displayName, passwordHash, role.
models/Answer.js	Saved answer schema with question, mood, settings, answer, metadata.

6. Complete Working Flow

A. Login flow

1. User opens /login.
2. Express renders views/login.ejs using res.render().
3. Frontend js/auth.js sends POST /api/login with username/email and password.
4. apiRoutes.js validates the body with Joi loginSchema.
5. authController.login calls userService.authenticateUser.
6. userService checks MongoDB through the User model and verifies password using bcrypt.
7. If valid, express-session stores user data in req.session.user.
8. Frontend redirects user to the home page.

B. Ask question flow

1. User selects mood, language, answer length, humor, study mode, focus mode, and enters question.
2. Frontend stores settings and opens /chat with query parameters.
3. Chat form sends POST /api/answer.
4. Joi validates the answer request.
5. requireAuth middleware checks active session.
6. answerController creates settings object and calls aiService.generateAnswer.
7. aiService builds the system instruction and user prompt for selected mode.
8. Gemini/OpenRouter returns structured answer text.
9. textParser converts text into answer, example, joke, supportive line, quick actions, and metadata.
10. historyService saves full answer data in MongoDB.
11. Frontend shows the answer in ChatGPT-style chat UI.

7. Database Design

The project uses MongoDB with Mongoose models. This makes data structure clear and easier to maintain.

Collection	Important Fields	Purpose
users	username, email, displayName, passwordHash, role	Stores login/signup users. Password is never stored in plain text.
answers	username, question, mood, language, answer, metadata, createdAt	Stores each generated answer so the user can view history later.

Answer model important fields:

- question - the user's question.
- mood - selected mood such as confused, stressed, curious, or tired.
- assistantMode - moodanswer, funtalk, rooms, or fungames.
- roomName/gameMode - optional context for specialized modes.
- answer, example, lightJoke, supportiveLine, quickActions - structured AI response.
- metadata - mood used, language used, theme tag, history title, safety level, feedback labels.

Interview line:

I used Mongoose because it gives schemas, validation at model level, indexes, and clean query methods. It also makes the database layer easier to scale compared to writing raw MongoDB queries everywhere.

8. Routes and API Endpoints

Method	Route	Purpose
GET	/login	Render login page.
GET	/signup	Render signup page.
GET	/	Render home page after authentication.
GET	/ask	Render ask/settings page.
GET	/chat, /fun, /room-chat, /game-chat	Render common chat page with different modes.
GET	/history	Render saved history page.
POST	/api/login	Validate credentials and create session.
POST	/api/signup	Validate signup form, create user, create session.
POST	/api/logout	Destroy session.
GET	/api/session	Check current login status.
POST	/api/answer	Generate AI answer and save history.
GET	/api/history	Return saved answers for logged-in user.

Interview note:

I used router.get and router.post inside separate route files instead of putting all app.get and app.post routes directly in app.js. This is still Express routing, but cleaner and more scalable.

9. AI Answer Generation Working

The AI service does three main jobs:

1. Selects the correct system prompt based on assistantMode: MoodAnswer, FunTalk, Rooms, or FunGames.
2. Builds a user prompt with question, mood, language, answer length, humor mode, supportive line, study mode, focus mode, room, game, and login mode.
3. Calls Gemini API first, or OpenRouter fallback if configured, then parses the structured output.

AI modes:

Mode	Behavior
MoodAnswer	Answers real questions with mood-based tone and safety rules.
FunTalk	Friendly casual chat, safe jokes, random fun questions, boredom relief.
AI Rooms	Different personalities like Study Room, Coding Help, Chill Cafe, Meme Zone.
FunGames	Safe interactive mini-games like quizzes, riddles, emoji game, rapid fire.

Structured response sections:

```
Answer:
[main answer]

Example:
[example or None]

Mood-friendly line:
[supportive line or None]

Optional light humor:
[safe joke or None]

Quick actions:
[action labels]

Metadata for app:
Mood used, Language used, Theme tag, History title, Safety level, Feedback
```

10. Validation, Security, and Error Handling

Area	Implementation	Why It Matters
Validation	Joi schemas in validators/	Bad form/API data is blocked before controllers run.
Authentication	express-session and requireAuth middleware	Protected pages and APIs require login.
Password security	bcrypt hashing in utils/password.js	Plain passwords are never stored.
Environment secrets	dotenv and .env	API keys and database URIs stay outside source code.
Error handling	404 and 500 middleware with error.ejs	Users see clean error pages instead of raw stack traces.
Logging	morgan	Requests are visible during development and debugging.

Validation example explanation:

When POST /api/answer is called, the request body first goes through answerSchema. Joi checks that question and mood exist, language is allowed, answerLength is valid, and boolean fields are converted properly. Only clean validated data reaches answerController.

11. Frontend and UI Features

The frontend is served from EJS pages and static assets inside public/.

- views/*.ejs render pages like login, signup, home, ask, chat, rooms, games, history, and settings.
- public/css contains feature-based CSS: global, navbar, hero, chat, mood, rooms, history, themes, responsive, animations.
- public/js contains browser logic: auth, chat, API calls, mood selection, voice, theme, history, settings, sound, streaks.
- Chat page uses a fixed bottom input like modern AI assistants.
- Voice button supports speech-to-text where browser support exists.
- Background sound mode is available for Chill Cafe, Late Night Talk, and Study Room.
- History page displays saved questions and opens full answer details.

Adaptive UI explanation:

The app changes visual style using mood and room context. For example, Study Room can use a focused theme, Chill Cafe can use a calm cafe vibe, and tired mood can use a softer low-energy theme. This makes the app feel more personalized.

12. Demo Script for Interview

Use this demo order during an interview:

1. Start the app with npm run dev and open http://localhost:3000.
2. Show login page and explain express-session plus bcrypt password hashing.
3. Login and open the home page.
4. Open Ask page, select mood as Confused, language as English, answer length as Medium, enable Study mode.
5. Ask: What is JavaScript?
6. Show chat page: user question appears, typing animation runs, AI answer appears below.
7. Explain that /api/answer validates with Joi, calls AI service, saves answer with Mongoose.
8. Open History page and show the saved question plus detail view.
9. Open Rooms page and show Study Room or Chill Cafe.
10. Explain future improvements: production session store, tests, deployment, and better analytics.

If interviewer asks why Express + EJS:

I chose EJS because it is simple, beginner-friendly, and good for server-rendered pages. Express handles routes and middleware cleanly, while static browser JS handles dynamic chat interactions.

13. Interview Questions and Answers

Question	Best Answer
Why did you use MVC?	MVC separates responsibilities. Routes define URLs, controllers handle request/response, services handle business logic, models handle database structure, and views render UI. This keeps code clean and scalable.
Why Joi instead of express-validator?	Joi gives reusable schema-based validation. It is easy to validate body and query data before controllers run.
How is authentication working?	User submits login form, Joi validates it, bcrypt verifies password, then express-session stores user data in req.session.user. Protected routes use requireAuth middleware.
How is AI response generated?	The frontend sends question and settings to /api/answer. The backend builds a prompt based on mood, room, and preferences, calls Gemini/OpenRouter, parses the structured response, saves it, and returns JSON.
How is history saved?	After successful AI generation, answerController calls historyService.saveAnswer, which stores question, settings, answer, metadata, and timestamp in MongoDB through the Answer model.
What was the hardest part?	Combining mood-aware prompt behavior, safe humor rules, multiple assistant modes, and clean MVC architecture while keeping the UI simple.

14. Commands and Environment

Install dependencies:

```
npm install
```

Run in development:

```
npm run dev
```

Run normally:

```
npm start
```

Important .env variables:

```
GEMINI_API_KEY=your_google_ai_studio_key
GEMINI_MODEL=gemini-2.5-flash
MONGODB_URI=your_mongodb_connection_string
MONGODB_DB=moodwise
SESSION_SECRET=long_random_secret
LOGIN_USERNAME=admin
LOGIN_PASSWORD=your_password
```

Do not expose .env in GitHub. Keep .env in .gitignore.

15. Final Summary

MoodAnswer AI is not only a simple chatbot. It is a multi-mode conversational platform with mood-aware answers, adaptive rooms, safe fun chat, mini-games, voice support, answer history, and an MVC Express backend.

Final 30-second answer:

MoodAnswer AI is my full-stack Node.js project. It uses Express and EJS for server-rendered pages, MongoDB and Mongoose for persistence, Joi for validation, bcrypt and sessions for authentication, and Gemini/OpenRouter for AI responses. The unique feature is mood-aware answering: the user selects mood and preferences, then the backend builds a safe structured prompt, generates an answer, saves it in history, and shows it in a modern chat UI. I followed MVC architecture so the project is clean, modular, and easy to scale.

Future improvements:

- Use MongoDB-backed session store for production.
- Add automated tests for auth, answer, history, and validation.
- Add delete/favorite history features with PUT/DELETE routes.
- Add analytics dashboard for moods, rooms, questions, and streaks.
- Deploy on Render/Railway with secure environment variables.